

# AI-Powered Smart Helmet

## Accident Prediction & Emergency Alert System

**College:** Dr. M.G.R. Educational and Research Institute

**Team:** Karthick S • Midhun Sathishkumar • Kamesh Kumar J

**Domain:** Artificial Intelligence • IoT • Machine Learning • Road Safety

**Type:** 50% Working Software Prototype with Simulated Sensor Data

### 1 Project Abstract

**Memorize this!** — This project presents an **AI-Powered Smart Helmet** that uses simulated IoT sensor data and a **Random Forest Machine Learning classifier** to predict rider safety conditions in real-time. The system monitors parameters like speed, vibration, alcohol level, drowsiness, acceleration, and gyroscope data. When a crash is predicted or rider impairment (alcohol/drowsiness) is detected, the system **locks the vehicle ignition, triggers alarms, and broadcasts GPS coordinates via emergency SOS SMS** to rescue services and family contacts — reducing the "Golden Hour" delay.

### 2 Problem Definition

| Problem              | Explanation                                                                                                            |
|----------------------|------------------------------------------------------------------------------------------------------------------------|
| Intoxicated Riding   | Alcohol impairs reflexes and motor control, leading to accidents                                                       |
| Drowsiness / Fatigue | Rider exhaustion causes micro-sleeps and swerving                                                                      |
| Golden Hour Delay    | Fatalities increase because emergency medical response is delayed — accidents happen without bystanders to report them |

**Key fact:** Motorcycle riders represent a disproportionately high fraction of overall traffic fatalities globally.

### **What is the "Golden Hour"?**

The **Golden Hour** refers to the first 60 minutes after a traumatic accident. Medical research shows that receiving treatment within this period significantly increases survival rates. Our project addresses this by **automatically detecting crashes** and **instantly sending GPS-tagged SOS alerts** — removing the dependency on bystanders.

### 3 Technology Stack

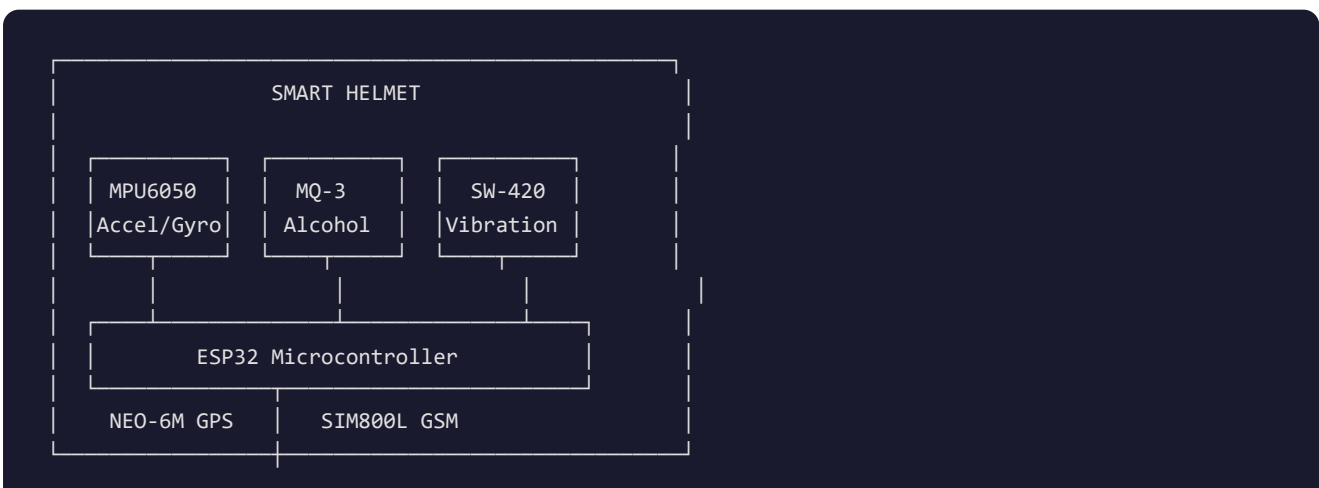
#### Backend

| Technology   | Purpose                                                      |
|--------------|--------------------------------------------------------------|
| Python       | Core programming language for backend                        |
| FastAPI      | Modern web framework to build REST API endpoints             |
| Uvicorn      | ASGI web server that runs the FastAPI app                    |
| scikit-learn | ML library — provides Random Forest Classifier               |
| joblib       | Serializes and saves/loads the trained ML model to disk      |
| pandas       | Data manipulation for creating synthetic training datasets   |
| numpy        | Numerical computing for feature arrays and random generation |

#### Frontend

| Technology   | Purpose                                                      |
|--------------|--------------------------------------------------------------|
| React 19     | JavaScript UI library for building the interactive dashboard |
| Vite         | Modern build tool & dev server (much faster than Webpack)    |
| Recharts     | Charting library for speed/vibration trend graphs            |
| Lucide-React | SVG icon library for clean UI icons                          |
| Vanilla CSS  | Custom dark-mode styling with glassmorphism and animations   |

### 4 System Architecture



| WiFi / HTTP



### FASTAPI BACKEND (Python)

Random Forest ML Classifier

10 features → 3 classes

80 trees, max depth 8



GET /api/live-data

GET /api/alert

POST /api/demo-mode

POST /api/reset-alert



| REST API (JSON)



### REACT FRONTEND (Vite)

Home Page | Live Dashboard | Project Modules

Sensor Cards | Charts | SOS Alert | GPS Map

## 5 Machine Learning Model — Deep Explanation

### Algorithm: Random Forest Classifier

**What is Random Forest?** It is an **ensemble learning** method that builds **multiple decision trees** during training. Each tree is trained on a random subset of data (called **bagging**). The final prediction is decided by **majority voting** — whichever class gets the most votes wins. It prevents overfitting better than a single decision tree.

### Model Configuration

| Parameter                 | Value | Meaning                                           |
|---------------------------|-------|---------------------------------------------------|
| <code>n_estimators</code> | 80    | Number of decision trees in the forest            |
| <code>max_depth</code>    | 8     | Maximum depth of each tree (prevents overfitting) |
| <code>random_state</code> | 42    | Seed for reproducibility                          |
| <code>test_size</code>    | 0.2   | 20% data for testing, 80% for training            |
| Training samples          | 1500  | Synthetic sensor data generated with numpy        |

### 10 Input Features

| #  | Feature                    | Unit             | Description                            | Range     |
|----|----------------------------|------------------|----------------------------------------|-----------|
| 1  | <code>speed</code>         | km/h             | Vehicle speed                          | 0–110     |
| 2  | <code>vibration</code>     | m/s <sup>2</sup> | Road vibration amplitude               | 0.2–18    |
| 3  | <code>alcohol_level</code> | mg/L             | Blood alcohol concentration            | 0.0–0.8   |
| 4  | <code>drowsy_score</code>  | 0–1              | Drowsiness / fatigue index             | 0.0–1.0   |
| 5  | <code>accel_x</code>       | G                | Lateral acceleration (left/right)      | -2 to 2   |
| 6  | <code>accel_y</code>       | G                | Longitudinal acceleration (front/back) | -2 to 2   |
| 7  | <code>accel_z</code>       | G                | Vertical acceleration (gravity ≈ 1G)   | 0.5–1.5   |
| 8  | <code>gyro_x</code>        | °/s              | Pitch (front/back tilt rate)           | -45 to 45 |
| 9  | <code>gyro_y</code>        | °/s              | Roll (side swerve rate)                | -45 to 45 |
| 10 | <code>gyro_z</code>        | °/s              | Yaw (heading rotation rate)            | -45 to 45 |

### 3 Output Classes

| Label | Class          | Meaning                                                         |
|-------|----------------|-----------------------------------------------------------------|
| 0     | <b>Safe</b>    | Rider is in normal safe condition                               |
| 1     | <b>At-Risk</b> | Warning — elevated speed, moderate alcohol, or unstable driving |
| 2     | <b>Danger</b>  | Critical — crash, heavily drunk, or severely drowsy             |

## Classification Rules

### 🔴 DANGER conditions:

- Vibration > 12.0 m/s<sup>2</sup> (crash impact)
- Acceleration X or Y exceeds ±3.0G (violent collision)
- Speed > 40 km/h AND vibration > 8.0 (high-speed crash)
- Alcohol level ≥ 0.35 mg/L (heavily intoxicated)
- Drowsy score ≥ 0.75 AND speed > 50 km/h (asleep while speeding)

### ⚠️ AT-RISK conditions:

- Speed > 80 km/h (overspeeding)
- Alcohol 0.15–0.35 (moderate alcohol)
- Drowsy score 0.50–0.75 (moderate drowsiness)
- Gyroscope X or Y > ±35°/s (unstable swerving)

### ✅ SAFE: All values within normal range

## How Prediction Works at Runtime

```
# 1. Collect sensor readings into a feature array
features = [speed, vibration, alcohol_level, drowsy_score,
            accel_x, accel_y, accel_z, gyro_x, gyro_y, gyro_z]

# 2. Feed into the trained model
prediction = model.predict([features])          # Returns: 0, 1, or 2
probabilities = model.predict_proba([features]) # Returns: [0.92, 0.05, 0.03]

# 3. Map to human-readable status
status_map = {0: "Safe", 1: "At-Risk", 2: "Danger"}
```

## Model Storage

The trained model is saved as `helmet_model.joblib` using the joblib library. On server startup, the model is loaded from disk (not retrained every time). If the file is missing, it auto-trains and saves a new one.

## 6 Backend API Endpoints

| Method | Endpoint                      | Purpose                                                                 |
|--------|-------------------------------|-------------------------------------------------------------------------|
| GET    | <code>/api/live-data</code>   | Returns current sensor data, ML prediction, and last 20 history entries |
| GET    | <code>/api/alert</code>       | Returns emergency alert status (crash active?, GPS coords, GSM status)  |
| POST   | <code>/api/demo-mode</code>   | Switches simulation: safe, risky, drunk, drowsy, crash                  |
| POST   | <code>/api/reset-alert</code> | Clears crash notification and resets to safe mode                       |
| GET    | <code>/docs</code>            | Auto-generated Swagger API documentation                                |

### Background Simulation Thread

- A **daemon thread** runs continuously in the background
- Every **1 second**, it generates new simulated sensor values based on the current demo mode
- Passes values through the **Random Forest model** for prediction
- Maintains a rolling **history buffer of 20 readings** for the charts

### What is CORS?

**CORS (Cross-Origin Resource Sharing)** is a browser security feature. Since frontend runs on `localhost:5173` and backend on `localhost:8080`, they are different "origins." Without CORS middleware, the browser would block API requests. We configure it in FastAPI to allow the frontend to communicate.

## 7 Five Simulation Demo Modes

| Mode          | Speed      | Alcohol   | Drowsy | Vibration | ML Result      |
|---------------|------------|-----------|--------|-----------|----------------|
| <b>Safe</b>   | 30–50 km/h | 0.01–0.05 | 0–15%  | 0.4–1.2   | ✅ Safe         |
| <b>Risky</b>  | 85–95 km/h | 0.04–0.11 | 15–28% | 1.8–3.5   | ⚠️ At-Risk     |
| <b>Drunk</b>  | 30–42 km/h | 0.38–0.45 | 8–25%  | 0.8–2.2   | 🔴 Danger       |
| <b>Drowsy</b> | 52–60 km/h | 0.0–0.04  | 82–94% | 0.6–1.6   | 🔴 Danger       |
| <b>Crash</b>  | 72→18→0    | 0.02      | 5%     | 18.7      | 🔴 Danger + SOS |

### **Crash Sequence (3 ticks):**

1. **Tick 1:** Riding normally at 72.4 km/h
2. **Tick 2:** Sudden deceleration to 18.2 km/h, vibration spikes to 18.7, acceleration hits 5.2G — **crash moment**
3. **Tick 3+:** Speed drops to 0.0, alert becomes active → **SOS SMS sent with GPS coordinates**



## 8 Hardware Components (Future Scope)

| Component               | Model         | Purpose                                                               |
|-------------------------|---------------|-----------------------------------------------------------------------|
| Accelerometer/Gyroscope | MPU6050       | Measures 3-axis acceleration (G-force) and angular velocity (°/s)     |
| Alcohol Sensor          | MQ-3          | Detects ethanol vapor from rider's breath inside helmet cavity        |
| Vibration Sensor        | SW-420        | Detects high-impact mechanical vibrations from collisions             |
| GPS Module              | NEO-6M        | Captures latitude/longitude coordinates via satellite signals         |
| GSM Module              | SIM800L       | Sends SMS over cellular network to emergency contacts                 |
| Buzzer                  | Active Buzzer | Sounds audible alarm inside helmet for drowsiness/alcohol warnings    |
| Microcontroller         | ESP32         | Central processing unit — reads sensors, runs logic, controls modules |

## 9 Project Modules (8 Modules)

| # | Module                   | Description                                                        |
|---|--------------------------|--------------------------------------------------------------------|
| 1 | Sensor Monitoring System | MPU6050, MQ-3, SW-420 sensors collecting data at high frequency    |
| 2 | AI/ML Risk Prediction    | Random Forest classifier — 10 features, 3 output classes, 80 trees |
| 3 | Alcohol Detection        | MQ-3 breath analysis, ignition lockout at BAC $\geq 0.35$          |
| 4 | Drowsiness Detection     | Eye-closure index monitoring, buzzer alarm at $\geq 75\%$ fatigue  |
| 5 | Crash & Impact Detection | G-force + vibration spike detection for collision events           |
| 6 | GPS Location Tracking    | NEO-6M satellite coordinate logging for accident location          |
| 7 | GSM Emergency SMS Alert  | SIM800L SMS dispatch with Google Maps crash link                   |
| 8 | Data Logging / Dashboard | Real-time React dashboard with charts, cards, and alerts           |

### Page 1: Home

- Hero section with project title and abstract
- Problem definition explanation
- Core features grid (ML, Emergency Alert, Impairment Detection, GPS)
- Project team section with member roles
- Future hardware integration flow diagram

### Page 2: Live Dashboard

- **Demo Mode Buttons:** Safe, Risky, Drunk, Drowsy, Crash
- **AI Prediction Card:** Shows SAFE/AT-RISK/DANGER with confidence bars
- **Sensor Cards:** Speed, Alcohol, Drowsiness, Vibration
- **3-Axis Accelerometer & Gyroscope:** X, Y, Z values
- **Emergency SOS Panel:** Crash severity, SMS status, Google Maps link
- **Trend Charts:** Real-time speed and vibration line charts

### Page 3: Project Modules

- Detailed breakdown of all 8 modules with technical specs
- Hardware future scope section

### Data Polling

Frontend polls the backend API **every 1 second** using `setInterval` and JavaScript `fetch()` API. New data triggers React state updates via `useState` and `useEffect` hooks, causing the UI to re-render.

## 11 Anticipated Viva Questions & Answers

### General Questions

#### Q1: What is the aim of your project?

*To develop an AI-Powered Smart Helmet that uses Machine Learning to predict accidents, detect rider impairment (alcohol/drowsiness), and automatically send emergency SOS alerts with GPS location to reduce the Golden Hour delay.*

#### Q2: What problem does this project solve?

*It solves three problems: (1) detecting drunk riding and locking the ignition, (2) detecting drowsy riding and sounding alarms, and (3) detecting crashes and immediately sending SMS with GPS coordinates — eliminating dependency on bystanders.*

#### Q3: Is this fully working hardware or software?

*This is a 50% working software prototype. The complete software stack — ML model, API backend, real-time dashboard — is fully functional. The hardware integration with actual sensors is planned for the next phase.*

#### Q4: Why did you choose this project?

*Motorcycle accidents are a leading cause of road fatalities. Many deaths happen not because of the accident itself, but because emergency help arrives too late. Our project automates crash detection and emergency alerts to save lives during the Golden Hour.*

### Machine Learning Questions

#### Q5: What ML algorithm did you use and why?

*We used Random Forest Classifier because: (1) it handles multi-class classification well, (2) it's robust against overfitting due to ensemble averaging, (3) it works well with tabular sensor data, and (4) it provides probability scores for confidence bars.*

#### Q6: How many features does your model use?

*10 features: speed, vibration, alcohol\_level, drowsy\_score, accel\_x, accel\_y, accel\_z, gyro\_x, gyro\_y, gyro\_z.*

**Q7: What are the output classes?**

*Three classes: Safe (label 0), At-Risk (label 1), and Danger (label 2).*

**Q8: How did you get the training data?**

*We generated 1500 synthetic training samples using numpy random functions with realistic sensor value ranges. Labels were assigned based on expert-defined rules — for example, vibration > 12.0 = Danger, speed > 80 = At-Risk.*

**Q9: What is the accuracy of your model?**

*The model achieves high accuracy (typically >95%) on the synthetic test set. In real-world deployment, accuracy would depend on actual sensor data calibration.*

**Q10: How many decision trees are in your forest?**

*80 decision trees (n\_estimators=80), each with a maximum depth of 8 (max\_depth=8).*

**Q11: What is overfitting? How does Random Forest prevent it?**

*Overfitting is when a model memorizes training data instead of learning general patterns. Random Forest prevents this by training multiple trees on random subsets of data (bagging) and averaging their predictions.*

**Q12: What is predict\_proba?**

*It returns the probability distribution across all classes. For example, [0.92, 0.05, 0.03] means 92% Safe, 5% At-Risk, 3% Danger. We display these as confidence bars in the dashboard.*

**Backend Questions****Q13: Why FastAPI instead of Flask?**

*FastAPI is faster, supports async/await natively, auto-generates Swagger docs at /docs, has built-in data validation using Pydantic, and is the modern standard for Python APIs.*

**Q14: What is CORS and why do you need it?**

*CORS (Cross-Origin Resource Sharing) is a browser security feature. Since frontend runs on localhost:5173 and backend on localhost:8080, they are different origins. Without CORS middleware, the browser would block API requests.*

**Q15: How does real-time data work?**

*A background daemon thread generates new sensor values every 1 second. The frontend polls the /api/live-data endpoint every 1 second using setInterval to fetch the latest values and re-render the UI.*

**Q16: What happens when a crash is detected?**

*The system: (1) detects high vibration ( $18.7 \text{ m/s}^2$ ) and G-force (5.2G), (2) drops speed to 0, (3) activates emergency alert with GPS coordinates, (4) simulates SOS SMS via GSM, and (5) generates a Google Maps link to the crash site.*

## Frontend Questions

### Q17: Why React?

*React is component-based, making it easy to build reusable UI elements. It efficiently updates only changed parts of the DOM using its virtual DOM — perfect for our dashboard that updates every second.*

### Q18: What is Vite?

*Vite is a modern build tool significantly faster than Webpack. It uses native ES modules and Hot Module Replacement (HMR) for instant updates during development.*

### Q19: How do the charts work?

*We use the Recharts library to render AreaCharts. The chart displays the last 20 historical data points (rolling buffer on the backend) for speed and vibration trends.*

## Hardware & Sensor Questions

### Q20: What is the MPU6050?

*It's a 6-axis sensor module with a 3-axis accelerometer (measures G-force) and a 3-axis gyroscope (measures angular velocity in °/s). Used to detect crashes, falls, and unstable riding.*

### Q21: How does the MQ-3 alcohol sensor work?

*The MQ-3 has a tin-dioxide ( $\text{SnO}_2$ ) sensitive layer. When ethanol vapor contacts this layer, its electrical resistance decreases proportionally to alcohol concentration. The analog output is read by the microcontroller.*

### Q22: What is the SW-420 vibration sensor?

*It's a high-sensitivity vibration sensor using a spring-based comparator circuit. When mechanical vibrations exceed a threshold, it outputs a digital HIGH signal — used to detect collisions.*

### Q23: How does GPS tracking work?

*The NEO-6M GPS module receives satellite signals and outputs NMEA formatted strings with latitude and longitude. These coordinates are parsed and included in the SOS SMS.*

**Q24: What is the SIM800L?**

*It's a GSM module that sends/receives SMS using a SIM card. We use AT commands to compose and send emergency SMS containing crash location and Google Maps links.*

**Architecture Questions****Q25: How does the frontend communicate with the backend?**

*Using JavaScript fetch() API to make HTTP requests. GET requests retrieve sensor data, POST requests change modes. Communication uses REST API with JSON format.*

**Q26: What is the complete data flow?**

*Sensor → ESP32 reads values → Sends to FastAPI via WiFi → Backend feeds into Random Forest → Predicts Safe/At-Risk/Danger → If crash detected → Activate alert → Send SMS via SIM800L with GPS → Dashboard shows SOS card.*

**Q27: Can this work with real hardware?**

*Yes! The software architecture is designed to accept real sensor data. Only the simulation code needs to be replaced with actual ESP32 hardware I/O — the API, ML model, and dashboard work unchanged.*

## 12 Advantages & Disadvantages

---

### Advantages

1. Real-time accident prediction using ML reduces emergency response time
2. Automated SOS eliminates dependency on bystanders
3. Alcohol detection prevents drunk riding via ignition lockout
4. Drowsiness alerts prevent fatigue-related accidents
5. GPS-tagged alerts enable precise location for ambulances
6. Modular architecture — easy to integrate real hardware later

### Limitations

1. Currently uses simulated sensor data, not real hardware
2. ML model trained on synthetic data — real accuracy may vary
3. Requires internet/cellular connectivity for SMS alerts
4. GPS accuracy depends on satellite visibility
5. Battery power management needs engineering for continuous operation

## 13 Future Enhancements

---

1. Physical hardware prototype with ESP32 + actual sensors
2. Cloud deployment on AWS/GCP for remote monitoring
3. Companion mobile app for family members
4. Deep learning models (LSTM/CNN) for better temporal recognition
5. Integration with hospital databases for direct ambulance dispatch
6. Voice alerts instead of just buzzer alarms
7. Multi-rider fleet monitoring dashboard for delivery companies

## 14 Key Technical Terms Glossary

---

| Term                 | Explanation                                                          |
|----------------------|----------------------------------------------------------------------|
| <b>Random Forest</b> | Ensemble ML algorithm — multiple decision trees with majority voting |
| <b>CORS</b>          | Browser security policy controlling cross-domain API access          |
| <b>REST API</b>      | Web service style using HTTP methods (GET, POST, PUT, DELETE)        |
| <b>FastAPI</b>       | Modern Python web framework with auto-docs and async support         |

---



|                             |                                                                 |
|-----------------------------|-----------------------------------------------------------------|
| <b>ASGI</b>                 | Async Server Gateway Interface — Python server protocol         |
| <b>Vite</b>                 | Next-gen JS build tool, faster than Webpack                     |
| <b>React</b>                | JS library for component-based UIs with virtual DOM             |
| <b>useState / useEffect</b> | React hooks for state management and side effects               |
| <b>scikit-learn</b>         | Python ML library with pre-built algorithms                     |
| <b>joblib</b>               | Python library for saving/loading ML models                     |
| <b>Synthetic Data</b>       | Artificially generated data mimicking real sensor readings      |
| <b>Feature Vector</b>       | Array of input values fed into an ML model                      |
| <b>Daemon Thread</b>        | Background thread that auto-stops when main program exits       |
| <b>Glassmorphism</b>        | UI design trend — frosted glass effect with blur & transparency |

## 15 Quick Revision Cheat Sheet

Project = AI-Powered Smart Helmet ML Model = Random Forest Classifier (80 trees, depth 8) Features = 10 (speed, vibration, alcohol, drowsy, accel×3, gyro×3) Classes = 3 (Safe, At-Risk, Danger) Training = 1500 synthetic samples, 80/20 train-test split Backend = FastAPI + Uvicorn (Python) on port 8080 Frontend = React + Vite (JavaScript) on port 5173 Charts = Recharts library Icons = Lucide-React Sensors = MPU6050, MQ-3, SW-420, NEO-6M GPS, SIM800L GSM MCU = ESP32 Demo Modes = Safe, Risky, Drunk, Drowsy, Crash Key APIs = /api/live-data, /api/alert, /api/demo-mode, /api/reset-alert

💡 **Pro tip for viva:** Always start with a one-line definition, then expand with how it's used in YOUR project. Example: *"Random Forest is an ensemble ML algorithm using multiple decision trees. In our project, we use 80 trees with max depth 8 to classify rider state into Safe, At-Risk, or Danger based on 10 sensor features."*